The Brilliant Employee · 02

A security alarm named a file that never existed

The Incident Files. The address was invented, the credential type was not. The real tokens were sitting in plaintext in my own chat logs. The pattern matched 47 of 15,110 transcript files, and reading them left two real credentials.

sgnk.ai/brilliant ↗

Sagnik Mitra

Why this exists

sgnk is a one-person studio. A language model writes a large share of the production code. You do not need your own system to get answers out of a model. You need one to find out when the answers were wrong, because wrong work reads exactly like right work and no prompt fixes that.

Three things from that week, none visible from inside the tool: a sizing step ignored on 796 of 816 tasks, two fixes I had written up as done that were never made, and a detector whose 1,112 catches were mostly one test string of my own.

THE SYSTEM, AS OF 6 AUGUST 2026, 17:29Z	
rules written	11 June, 56 days ago
ledger running	27 June, 40 days
tasks logged	3,065, across 3,564 rows
controls on tool calls	**6 registered, 3 of them armed**

You can rent the model. You cannot rent the part that tells you it was wrong.

Five parts, and where today sits

Five parts sit around the model. What changes from one piece of writing to the next is which of them is under the microscope.

sizing check	sizes a job before a model is picked
model picker	recommends which model gets the job
safety stops	refuse the irreversible, before it runs
grader	grades finished work pass or fail
task log	at least one append-only line per task

Listed in the order they act on a task, not the order they were built.

Today cuts across more than one of them, so no row is marked. Nothing below is assumed knowledge; everything this post needs, it explains.

> **ASIDE.** Only the safety stops can refuse. The sizing check advises, the model picker recommends, the grader grades after the fact, and the task log only records. Four of the five observe; one intervenes.

This page is the map. Nothing here needs another post to make sense.

Detail arrives as fast as truth does

One of my automation system's audit subagents came back from a repository sweep with a specific, confident finding: leaked personal access tokens, sitting in testing/workspaces.json.

> **Plain words: PERSONAL ACCESS TOKEN.** a long random string that stands in for your password when a PROGRAM talks to a service. Anyone holding one is you, as far as that service can tell. GitHub, Vercel, Supabase and Cloudflare each issue their own.

WHAT THE FINDING CARRIED	
the path named	**testing/workspaces.json**
the credential named	personal access token
the stakes	whoever holds it is you

This is exactly the class of alarm you cannot sit on, and exactly the class you most want to forward untouched, which is the trap. Specific claims feel pre-verified. They are not.

Specificity is not evidence. It is just detail.

The named file never existed

Before repeating the finding anywhere, I went to the file. There is no testing/workspaces.json. Not moved, not renamed, not deleted in some recent commit. The path had never existed.

EXIT ONE

do

close it as a hallucination

EXIT TWO

do

soften it to a possible leak

Both exits end the search at exactly the moment it should widen. They are the same mistake in opposite directions, treating a subordinate's alarm as a verdict rather than a hypothesis.

I nearly took the first one. An alarm with a wrong address reads like a false alarm. It is not the same thing.

> **ASIDE.** The address was never the load-bearing part of the alarm. The credential type was.

A wrong alarm can still be pointing at a real fire.

A dashboard needs a human. A cron job does not.

The choice is not between a token and a nicer workflow. It is between interactive and unattended.

USE THE VENDOR'S LOGIN	YOU NEED A RAW TOKEN
deploy by hand **the dashboard**	a scheduled job **nobody to click Approve**
your own terminal **CLI login, stores a credential**	a build elsewhere **no browser to redirect to**
the vendor's own CI **built-in token, nothing to hold**	a script an agent runs **no human in the loop**

A token is the only identity a program can present on its own. That makes the question not whether to use one, but how to hold it.

> **ASIDE.** GitHub's own guidance points the same way: use the GitHub CLI or Git Credential Manager on the command line, the built-in token inside its own Actions, and an app for a long-lived integration. A personal token is what is left over.

Reach for a token when nobody is there to approve. Not before.

You cannot stop a leak. You can make it cheap.

1. Fine-grained, scoped to the one repository or resource the job touches.
2. Shortest expiry you can live with, plus an IP restriction where offered.
3. Value in one file outside every repository, sourced by the command that needs it.
4. Never echo it. Rotate on doubt, not on proof.

Rotation is three moves: create the new one, replace it everywhere, delete the old. The third is the one people skip.

One of my two real leaks was an environment variable a tool echoed into its own output. It was held exactly as recommended. Storage is a probability; expiry is a ceiling.

> **ASIDE.** An environment variable stops the value being committed, shared in a file, or read off the repository. It does not stop a shell echoing a command, a debug print, a crash dump, or a transcript recording the terminal.

Scope and expiry beat storage, because storage fails quietly.

Widen from the path to the pattern

1. Alarm raised: leaked tokens at a specific named path
2. Named path checked live: the file does not exist
3. Search widened from the path to the pattern: token-shaped strings, anywhere

```
# the alarm named a path
$ find . -path "*testing/workspaces.json"
$

# widen from path to pattern
$ grep -rlE 'gh[pousr]_[A-Za-z0-9]{20,}' \
    ~/.claude/projects | wc -l
  (a two-digit number, and rising)
```

The pivot is step three. Stop auditing the claim and start auditing the world.

Verify the claim. Then search wider than the pointer.

The leak was in my chat logs

Every session my automation system runs is written to a log file on my own disk, line by line, as it happens. The vendor's own documentation calls that file plaintext, and the same document says known key and token patterns are redacted before a shared session is uploaded. The redactor runs on the copy that leaves.

WHERE THE ALARM POINTED	WHERE THE FIRE WAS
path **testing/workspaces.json**	path **~/.claude/projects/**.jsonl**
exists **no**	what **my own session transcripts**
tokens found **0**	tokens found **GitHub-token-shaped strings, plaintext**

ASIDE. A public request to scrub the file that stays was filed against the claude-code repository and closed as not planned. A decision, not a blind spot.

The most dangerous file on the machine is the one that writes itself.

Every vendor already told you not to do this

PRINTED ON THE PAGE THAT ISSUES THE TOKEN	
Cloudflare	shown once; never in plaintext
Supabase	never over chat or email
GitHub	never hardcode; scope it, expire it
Vercel	returned once, never retrievable again
your AI coding tool	**writes the whole chat to disk, plaintext**

A session is a chat, and it is written to disk in the clear. So a tool that helps you work breaks two of those three rules automatically, for every session, without anyone deciding to.

> **ASIDE.** Vercel's token record carries fields named leakedAt and leakedUrl: when it was marked leaked, and where it was found. The platform models a leak as a normal event, not an exception.

The advice was right. It just predates a tool that logs everything you type.

The lesson lives in the rulebook, not the worker

LEARNED RULE #2, STANDING RULEBOOK	
seeded	2026-06-27
trigger	any security or factual claim from a subagent
step one	**verify it live before repeating it as fact**
step two	attribute anything you could not verify
symmetry	never dismiss on a wrong pointer, widen the search
Append-only. The incident happened once. The rule loads every session.	

The rulebook is the actual workplace. The system reads it at the start of every session, so the cost of the incident was paid once and the lesson compounds daily. That is the trade this whole series is about: the reliability lives in the paper around the employee, not in the employee.

An incident you survived is worth exactly what you wrote down.

Count distinct strings, not grep hits

1. Never repeat it as fact before a live check
2. Never dismiss it on a wrong pointer. Wrong address, real fire is a common shape
3. Widen from the named location to the pattern

47 of 15,110 files — matched the pattern

163 occurrences — of 20 distinct strings

7 at 40 characters — the length a real one is

Two real credentials — what reading the 7 left

> **ASIDE.** Re-derived 2026-08-04 22:54Z, pattern gh[pousr]_ then 20 or more characters.

Alarms are hypotheses. Fires are facts.
A live check is the only converter.

Nobody is going to clean the AI chat logs on your disk.

Run the pattern on your own machine and count distinct strings, not hits. Full method and the exact commands.

sgnk.ai/brilliant ↗

Sagnik Mitra

I build AI systems and the machinery that keeps them honest: gates, ledgers, judges, and rails. Product design for years, engineering the whole time. This series is what the building actually taught me.

`sgnk.ai · in/sagnikmitra · github.com/sagnikmitra`